Part 23: Feature Selection and Temporal Validation

Financial Data Science II Updated March 17, 2026

Part 23: Feature Selection and Temporal Validation

Part 22 focused on the construction of candidate predictors. The next task is to determine which of those predictors are worth keeping.

In financial modelling this step cannot be reduced to a single ranking rule. A feature may look useful in sample and fail out of sample. A feature may also be statistically related to the target but impossible to compute in real time.

For that reason, feature selection is best viewed as a sequence of screens rather than as a single algorithm.

Exercise. Explain the difference between the following statements.

1. A feature is correlated with the target in sample.
2. A feature is useful in a predictive model.

Basic Terms

We use the following terms in this Part.

A **screening rule** is a preliminary rule used to reduce a large candidate set. A **selection rule** determines which features are kept for modelling. **Redundancy** refers to overlap in the information carried by different features. **Stability** refers to whether a feature behaves similarly across periods or regimes. An **ablation study** compares a baseline specification to an expanded one in order to test the incremental value of a feature or feature family.

Finally, we distinguish between a **ranking statistic** and a **modelling objective**. For example, a feature may be ranked by a univariate F-score or by mutual information, while the final predictive model is judged by out-of-sample RMSE, R^2, AUC, or some other metric.

Exercise. Why is it possible for two features to rank highly on their own but add little value when used together in the same model?

Why Feature Selection is Needed

Feature selection matters for at least four reasons.

1. Large feature sets can lead to unstable fitted models.
2. Many features are redundant transformations of the same underlying signal.
3. Financial relationships vary over time, so a feature that works in one period may fail in another.
4. Some features are costly to maintain or impossible to compute in a production setting.

The goal is therefore not to maximize the number of features. The goal is to keep the subset that is both informative and usable.

A Selection Framework

A practical sequence is the following.

1. Remove features that are infeasible or contaminated by leakage.
2. Remove features with too many missing values or too little variation.
3. Rank the remaining features using simple univariate measures.
4. Check whether the ranking is stable across time.
5. Evaluate incremental predictive value using walk-forward validation.

Univariate Screening

In this Part we use two simple ranking statistics.

1. **F-score:** emphasizes linear association with a continuous target.
2. **Mutual information:** allows for more general dependence.

A convenient summary is the **combined rank**, defined as the average of the rank under each criterion. This is a screening device, not the final objective function.

Example A: Daily Feature Screening

We return to the daily equity setting from Part 22. For illustration, we construct a modest feature set from prices and rough accounting ratios. We then rank the features and compare predictive performance as the feature set expands.

```
import warnings
warnings.filterwarnings('ignore')

from pathlib import Path
import zipfile

import numpy as np
import pandas as pd
import yfinance as yf

from sklearn.feature_selection import f_regression, mutual_info_regression
from sklearn.linear_model import Ridge, LogisticRegression
from sklearn.metrics import mean_squared_error, r2_score, roc_auc_score, balanced_accuracy_s
from sklearn.model_selection import TimeSeriesSplit
```

```
TICKER = 'AAPL'
START = '2016-01-01'
END = '2026-03-01'

px = yf.download(TICKER, start=START, end=END, auto_adjust=False, progress=False)
if isinstance(px.columns, pd.MultiIndex):
    px.columns = px.columns.get_level_values(0)
px.columns = [str(c).lower().replace(' ', '_') for c in px.columns]
if 'adj_close' not in px.columns and 'adj close' in px.columns:
    px['adj_close'] = px['adj close']
if 'adj_close' not in px.columns:
    px['adj_close'] = px['close']

daily = pd.DataFrame(index=px.index)
daily['ret_1d'] = px['adj_close'].pct_change()
daily['log_ret_1d'] = np.log(px['adj_close'] / px['adj_close'].shift(1))
daily['mom_5d'] = px['adj_close'].pct_change(5)
daily['mom_21d'] = px['adj_close'].pct_change(21)
daily['mom_63d'] = px['adj_close'].pct_change(63)
daily['vol_21d'] = daily['ret_1d'].rolling(21).std()
daily['vol_63d'] = daily['ret_1d'].rolling(63).std()
daily['drawdown'] = px['adj_close'] / px['adj_close'].cummax() - 1
```

```
def get_financial_table(ticker, attr):
    obj = getattr(ticker, attr, None)
    return obj if isinstance(obj, pd.DataFrame) else pd.DataFrame()


def get_shares_series(ticker):
    shares = None
```

```
    for getter in ('get_shares_full', 'get_shares'):
        try:
            shares = getattr(ticker, getter)()
            if shares is not None:
                break
        except Exception:
            continue
    if isinstance(shares, pd.DataFrame) and shares.shape[1] > 0:
        shares = shares.iloc[:, 0]
    if shares is None or not hasattr(shares, 'index'):
        return None
    shares = shares[~shares.index.duplicated(keep='last')]
    shares.index = pd.to_datetime(shares.index, errors='coerce').tz_localize(None)
    return shares.dropna()


def build_per_share(statement_df, shares, names):
    if statement_df.empty or shares is None or len(shares) == 0:
        return None
    statement_df = statement_df.T.copy()
    statement_df.index = pd.to_datetime(statement_df.index, errors='coerce').tz_localize(Nor
    chosen = None
    for col in statement_df.columns:
        if any(name in str(col) for name in names):
            chosen = col
            break
    if chosen is None:
        return None
    out = (statement_df[chosen] / shares.reindex(statement_df.index, method='nearest')).repl
    if isinstance(out, pd.DataFrame) and out.shape[1] > 0:
        out = out.iloc[:, 0]
    return out


tk = yf.Ticker(TICKER)
income_q = get_financial_table(tk, 'quarterly_income_stmt')
if income_q.empty:
    income_q = get_financial_table(tk, 'income_stmt')
balance_q = get_financial_table(tk, 'quarterly_balance_sheet')
if balance_q.empty:
    balance_q = get_financial_table(tk, 'balance_sheet')

shares = get_shares_series(tk)
eps_q = build_per_share(income_q, shares, ['Net Income'])
bvps_q = build_per_share(balance_q, shares, ['Total Stockholder Equity', 'Total Equity'])

if eps_q is not None:
    daily['pe'] = px['adj_close'] / eps_q.reindex(daily.index, method='ffill')
if bvps_q is not None:
    daily['pb'] = px['adj_close'] / bvps_q.reindex(daily.index, method='ffill')

daily['target_fwd_ret_5d'] = px['adj_close'].shift(-5) / px['adj_close'] - 1
data_daily = daily.replace([np.inf, -np.inf], np.nan).dropna().copy()

X_daily = data_daily.drop(columns=['target_fwd_ret_5d'])
```

```
y_daily = data_daily['target_fwd_ret_5d']
X_daily.head()
```

	ret_1d	log_ret_1d	mom_5d	mom_21d	mom_63d	vol_21d	vol_63d	drawdo
Date								
2024-10-01	-0.029142	-0.029575	-0.005102	-0.012183	0.028156	0.015860	0.014754	-0.035
2024-10-02	0.002520	0.002517	0.001811	0.018001	0.024791	0.014633	0.014741	-0.033
2024-10-03	-0.004895	-0.004907	-0.008131	0.021825	-0.001806	0.014533	0.014505	-0.037
2024-10-04	0.005007	0.004995	-0.004346	0.019876	-0.003325	0.014501	0.014495	-0.033
2024-10-07	-0.022531	-0.022788	-0.048541	0.003940	-0.029445	0.015305	0.014762	-0.054

```
f_scores, _ = f_regression(X_daily, y_daily)
mi_scores = mutual_info_regression(X_daily, y_daily, random_state=42)

ranking_daily = pd.DataFrame({
    'feature': X_daily.columns,
    'f_score': f_scores,
    'mi_score': mi_scores,
})
ranking_daily['rank_f'] = ranking_daily['f_score'].rank(ascending=False, method='average')
ranking_daily['rank_mi'] = ranking_daily['mi_score'].rank(ascending=False, method='average')
ranking_daily['combined_rank'] = (ranking_daily['rank_f'] + ranking_daily['rank_mi']) / 2
ranking_daily = ranking_daily.sort_values('combined_rank').reset_index(drop=True)
ranking_daily
```

	feature	f_score	mi_score	rank_f	rank_mi	combined_rank
0	pb	28.372708	0.259668	1.0	2.0	1.5
1	pe	12.422415	0.270763	3.0	1.0	2.0
2	drawdown	7.921998	0.211750	4.0	3.0	3.5
3	mom_21d	7.029405	0.091762	5.0	5.0	5.0
4	mom_5d	18.259151	0.000000	2.0	10.0	6.0
5	vol_21d	0.676207	0.144032	8.0	4.0	6.0
6	ret_1d	0.934216	0.062242	7.0	8.0	7.5
7	log_ret_1d	0.951107	0.061004	6.0	9.0	7.5
8	mom_63d	0.003459	0.091008	10.0	6.0	8.0
9	vol_63d	0.592643	0.089446	9.0	7.0	8.0

The table above is a ranking table, not a final verdict. It is useful because it provides a simple way to reduce the feature set before more expensive validation. It is limited because it ignores interactions, redundancy, and time variation.

```
def walk_forward_regression(X, y, model, n_splits=5):
    tscv = TimeSeriesSplit(n_splits=n_splits)
    rows = []
    for fold, (train_idx, test_idx) in enumerate(tscv.split(X), start=1):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        rows.append({
            'fold': fold,
            'rmse': mean_squared_error(y_test, pred) ** 0.5,
            'r2': r2_score(y_test, pred),
            'sign_accuracy': (np.sign(pred) == np.sign(y_test)).mean(),
        })
    return pd.DataFrame(rows)


top_counts = [3, 5, min(8, X_daily.shape[1]), X_daily.shape[1]]
top_counts = sorted(set(top_counts))
selection_summary = []

for k in top_counts:
    chosen = ranking_daily['feature'].iloc[:k].tolist()
    scores = walk_forward_regression(X_daily[chosen], y_daily, Ridge(alpha=1.0))
    selection_summary.append({
        'feature_set': f'top_{k}',
        'rmse': scores['rmse'].mean(),
        'r2': scores['r2'].mean(),
        'sign_accuracy': scores['sign_accuracy'].mean(),
    })

pd.DataFrame(selection_summary).sort_values('rmse').reset_index(drop=True)
```

	feature_set	rmse	r2	sign_accuracy
0	top_5	0.046920	-0.308406	0.473077
1	top_8	0.046930	-0.308711	0.473077
2	top_3	0.047044	-0.318993	0.484615
3	top_10	0.049710	-0.525360	0.480769

Exercise. Why might the feature set with the best univariate ranking fail to produce the best multivariate model?

Stability Across Time

A feature that ranks well over the full sample may do so only because of one favorable period. It is therefore useful to check whether a feature retains its sign, magnitude, or rank across rolling subsamples.

```
windows = []
split_points = np.array_split(np.arange(len(X_daily)), 4)

for window_id, idx in enumerate(split_points, start=1):
    X_window = X_daily.iloc[idx]
    y_window = y_daily.iloc[idx]
    if len(X_window) < 50:
        continue
    f_scores_w, _ = f_regression(X_window, y_window)
    mi_scores_w = mutual_info_regression(X_window, y_window, random_state=42)
    temp = pd.DataFrame({
        'feature': X_daily.columns,
        'window': window_id,
        'f_score': f_scores_w,
        'mi_score': mi_scores_w,
    })
    temp['rank_f'] = temp['f_score'].rank(ascending=False, method='average')
    temp['rank_mi'] = temp['mi_score'].rank(ascending=False, method='average')
    temp['combined_rank'] = (temp['rank_f'] + temp['rank_mi']) / 2
    windows.append(temp[['feature', 'window', 'combined_rank']])

rank_stability = pd.concat(windows, ignore_index=True)
stability_summary = rank_stability.groupby('feature')['combined_rank'].agg(['mean', 'std']).
stability_summary
```

	mean	std
feature		
pe	2.3750	1.108678
drawdown	3.6250	1.108678
pb	3.6250	1.796988
vol_21d	4.8750	2.250000
vol_63d	5.0000	2.345208
mom_21d	5.1250	2.495830
mom_63d	5.6250	1.887459
mom_5d	6.6250	2.056494
log_ret_1d	8.9375	0.718070
ret_1d	9.1875	0.850857

Example B: LOB Feature Screening

We now repeat the exercise in the LOB setting. The point is not to build a definitive HFT model. The point is to show how simple screening and ablation ideas transfer to event-driven data.

```
def locate_lobster_sample():
    candidates = [
        Path('/Users/harold/4. RA work/FACTOR_TRAINING_CHI/lobster_samples/AAPL_2012-06-21_5
        Path('/Users/harold/Documents/LOBSTER_SampleFile_AAPL_2012-06-21_5.zip'),
    ]
    for candidate in candidates:
        if candidate.is_dir():
            return candidate
        if candidate.is_file() and candidate.suffix == '.zip':
            extract_dir = Path('/Users/harold/4. RA work/Factor_Training_Eng/.cache') / canc
            extract_dir.mkdir(parents=True, exist_ok=True)
            with zipfile.ZipFile(candidate) as zf:
                zf.extractall(extract_dir)
            return extract_dir
    raise FileNotFoundError('Could not locate the AAPL LOBSTER sample directory or zip file.


lobster_dir = locate_lobster_sample()
msg_path = next(lobster_dir.glob('*message_5.csv'))
book_path = next(lobster_dir.glob('*orderbook_5.csv'))

msg_cols = ['time', 'type', 'order_id', 'size', 'price', 'direction']
book_cols = []
for level in range(1, 6):
    book_cols += [f'ask_{level}', f'ask_size_{level}', f'bid_{level}', f'bid_size_{level}']

msg = pd.read_csv(msg_path, header=None, names=msg_cols)
book = pd.read_csv(book_path, header=None, names=book_cols)
lob = pd.concat([msg, book], axis=1)


def compute_ofi_one_level(bid_price, bid_size, ask_price, ask_size):
    bid_prev = bid_price.shift(1)
    ask_prev = ask_price.shift(1)
    bid_size_prev = bid_size.shift(1)
    ask_size_prev = ask_size.shift(1)
    bid_contrib = np.where(bid_price > bid_prev, bid_size, np.where(bid_price == bid_prev, b
    ask_contrib = np.where(ask_price < ask_prev, ask_size, np.where(ask_price == ask_prev, a
    return pd.Series(bid_contrib - ask_contrib, index=bid_price.index)


lob_feat = lob.copy()
lob_feat['mid'] = (lob_feat['bid_1'] + lob_feat['ask_1']) / 2
lob_feat['spread_bps'] = 10000 * (lob_feat['ask_1'] - lob_feat['bid_1']) / lob_feat['mid']
lob_feat['imbalance_l1'] = (lob_feat['bid_size_1'] - lob_feat['ask_size_1']) / (lob_feat['bi
bid_depth_5 = lob_feat[[f'bid_size_{i}' for i in range(1, 6)]].sum(axis=1)
ask_depth_5 = lob_feat[[f'ask_size_{i}' for i in range(1, 6)]].sum(axis=1)
lob_feat['imbalance_l5'] = (bid_depth_5 - ask_depth_5) / (bid_depth_5 + ask_depth_5)
```

```
lob_feat['microprice'] = (lob_feat['ask_1'] * lob_feat['bid_size_1'] + lob_feat['bid_1'] * l
lob_feat['microprice_dev_bp'] = 10000 * (lob_feat['microprice'] - lob_feat['mid']) / lob_fea
lob_feat['ofi_l1'] = compute_ofi_one_level(lob_feat['bid_1'], lob_feat['bid_size_1'], lob_fe
lob_feat['ofi_l5'] = sum(
    compute_ofi_one_level(lob_feat[f'bid_{level}'], lob_feat[f'bid_size_{level}'], lob_feat
    for level in range(1, 6)
)
lob_feat['signed_size'] = lob_feat['direction'] * lob_feat['size']
lob_feat['time_floor'] = np.floor(lob_feat['time']).astype(int)

bars = (
    lob_feat.groupby('time_floor', sort=True)
    .agg(
        spread_bps=('spread_bps', 'last'),
        imbalance_l1=('imbalance_l1', 'last'),
        imbalance_l5=('imbalance_l5', 'last'),
        microprice_dev_bp=('microprice_dev_bp', 'last'),
        ofi_l1=('ofi_l1', 'sum'),
        ofi_l5=('ofi_l5', 'sum'),
        signed_size=('signed_size', 'sum'),
        msg_count=('type', 'size'),
        mid=('mid', 'last'),
    )
    .reset_index()
)
bars['target_up_1s'] = (bars['mid'].shift(-1) > bars['mid']).astype(int)
bars = bars.dropna().copy()
bars.head()
```

	time_floor	spread_bps	imbalance_l1	imbalance_l5	microprice_dev_bp	ofi_l1	ofi_l5
0	34200	2.219168	0.000000	-0.673647	0.000000	82.0	-2199.0
1	34201	5.122776	0.000000	0.111111	0.000000	-326.0	-3045.0
2	34202	3.586219	0.694915	0.333333	1.246059	-144.0	-1566.0
3	34203	4.440042	-0.694915	-0.166861	-1.542727	70.0	1200.0
4	34204	3.073823	-0.960784	-0.619938	-1.476640	-718.0	173.0

```
feature_cols_lob = ['spread_bps', 'imbalance_l1', 'imbalance_l5', 'microprice_dev_bp', 'ofi_


def walk_forward_auc(feature_frame, y, feature_names, n_splits=5):
    tscv = TimeSeriesSplit(n_splits=n_splits)
    aucs = []
    baccs = []
    for train_idx, test_idx in tscv.split(feature_frame):
        X_train = feature_frame.iloc[train_idx][feature_names]
        X_test = feature_frame.iloc[test_idx][feature_names]
        y_train = y.iloc[train_idx]
        y_test = y.iloc[test_idx]
        model = LogisticRegression(max_iter=500)
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
```

```
        prob = model.predict_proba(X_test)[:, 1]
        aucs.append(roc_auc_score(y_test, prob))
        baccs.append(balanced_accuracy_score(y_test, pred))
    return float(np.mean(aucs)), float(np.mean(baccs))


lob_univariate = []
y_lob = bars['target_up_1s']
for feature in feature_cols_lob:
    auc, bacc = walk_forward_auc(bars[feature_cols_lob], y_lob, [feature])
    lob_univariate.append({'feature': feature, 'auc': auc, 'balanced_accuracy': bacc})

lob_univariate = pd.DataFrame(lob_univariate).sort_values('auc', ascending=False).reset_inde
lob_univariate
```

	feature	auc	balanced_accuracy
0	msg_count	0.554703	0.501571
1	imbalance_l1	0.538607	0.500000
2	microprice_dev_bp	0.534970	0.500000
3	ofi_l1	0.524367	0.500130
4	spread_bps	0.506381	0.500000
5	imbalance_l5	0.502624	0.500000
6	ofi_l5	0.500580	0.500000
7	signed_size	0.494757	0.499957

```
feature_sets = {
    'base_book_state': ['spread_bps', 'imbalance_l1'],
    'book_plus_depth': ['spread_bps', 'imbalance_l1', 'imbalance_l5', 'microprice_dev_bp'],
    'book_plus_ofi': ['spread_bps', 'imbalance_l1', 'imbalance_l5', 'microprice_dev_bp', 'of
    'full_set': feature_cols_lob,
}

lob_set_summary = []
for name, cols in feature_sets.items():
    auc, bacc = walk_forward_auc(bars[feature_cols_lob], y_lob, cols)
    lob_set_summary.append({'feature_set': name, 'auc': auc, 'balanced_accuracy': bacc})

pd.DataFrame(lob_set_summary).sort_values('auc', ascending=False).reset_index(drop=True)
```

	feature_set	auc	balanced_accuracy
0	full_set	0.558731	0.503890
1	book_plus_ofi	0.539412	0.500062
2	book_plus_depth	0.534492	0.500000
3	base_book_state	0.528238	0.500000

The LOB example makes the logic of ablation particularly clear. One may begin with a minimal state description of the best bid and ask, then add depth, then add OFI, and ask whether each expansion produces an out-of-sample improvement.

What Should Be Reported?

When presenting a selected feature set, it is useful to report at least the following.

1. The original candidate features.
2. The screening criteria.
3. The final selected subset.
4. The validation scheme.
5. The metric by which performance is judged.
6. The incremental contribution of the selected features relative to a simpler baseline.

This makes it easier for a reader to distinguish between a feature that is merely interesting and a feature that is useful for a forecasting problem.

Exercises

Exercise 1. Give an example of a feature that ranks well by mutual information but might still be rejected for practical reasons.

Exercise 2. Suppose that two features have nearly identical combined rank values. What additional information would you want before choosing between them?

Exercise 3. In the LOB setting, why is it useful to compare a base book-state model to a base-plus-OFI model rather than jumping immediately to the full feature set?

Exercise 4. Explain why a feature-selection rule based on a shuffled train/test split is not appropriate in this setting.

References

1. Cont, Kukanov, and Stoikov (2013), *The Price Impact of Order Book Events*.
2. Guyon and Elisseeff (2003), *An Introduction to Variable and Feature Selection*.
3. Lopez de Prado (2018), *Advances in Financial Machine Learning*.
4. LOBSTER documentation and sample files.